

## CSCE 678 DISTRIBUTED SYSTEMS AND CLOUD COMPUTING · MODULE 11 · LECTURE NOTES

# Trust, Cheating, and Fairness

*Byzantine participants, and a referee that cannot see everything.*

## MODULE OUTCOMES

- Apply the authoritative-server principle consistently.
- Explain lag compensation and the unfairness it creates.
- Classify cheats by what they exploit.
- Explain why information hiding beats detection.
- Reason about anti-cheat trade-offs honestly.

## 1 The authoritative server principle

### THE CLIENT MAY REPORT INTENT, NEVER OUTCOME

'I am holding the forward key' is **intent**. The server decides whether that produces movement, how much, and whether a wall intervenes.

'I have moved to position (x, y, z)' is an **outcome**, and a server that accepts it has shipped a teleport hack — the client simply reports a different position.

'I fired at this angle on tick 4821' is **intent**. 'I hit the enemy for 40 damage' is an **outcome**, and accepting it is an aimbot that does not even need to aim — the cheat does not have to hit anything, only to claim it did.

**Every single exploit in every game is this rule being broken somewhere** — and it is almost never broken out of ignorance. It is broken for a good reason, under a deadline, to fix a specific feel problem or save specific server CPU, with a comment saying it will be revisited. **So the useful discipline is not knowing the rule but auditing for where it has been relaxed**, which should be a recurring and deliberate exercise rather than a one-time design decision.

- **Is this movement physically possible** from the last validated position within the elapsed time, given the player's speed, the terrain, and their current state? Catches teleporting and most flying.
- **Is the action permitted at all** — is the weapon actually owned, is the ability off cooldown, is the player alive, do they have the ammunition, are they in a state that allows it?
- **Is the input rate plausible?** Inputs arriving faster than real time are a speedhack, and the check is a simple comparison against the wall clock with a tolerance for jitter.
- **Is the target reachable** — line of sight, within range, and not through geometry? This requires a server-side world representation, which is a real cost and is the reason some games skip it.

- **And is the claimed tick number sane** — not in the future, not implausibly far in the past (which would exploit lag compensation, §2), and not one already processed. **These checks collectively are the entire security model**; everything in §4 is making them harder to circumvent rather than replacing them.

## 2 Lag compensation

---

### THE PROBLEM

the shooter sees the target where it was (their latency plus interpolation delay) ago -- so aiming at what they SEE and firing MISSES, because the server has since moved the target. Shots that visually connect are scored as misses.

### LAG COMPENSATION

the server keeps a HISTORY of every player's position, one entry per tick, for about a second.  
when a shot arrives tagged with tick N:  
    REWIND every other player to their tick-N position  
    hit-test against that rewound world  
    restore the present  
-> the shot hits what the shooter actually saw.

### THE COST -- "SHOT AROUND A CORNER"

the target has since moved behind cover. On THEIR screen they are safe. They take damage anyway, because at tick N they were not safe.

BOTH PLAYERS EXPERIENCED THE GAME CORRECTLY AND THEY DISAGREE.

**That last line is the point, and it is worth stating plainly.** Neither player is wrong, neither client is buggy, and no amount of engineering removes the disagreement. **It is the irreducible cost of a shared world with no shared present** (Module 03 §1) — the same impossibility that made logical clocks necessary, surfacing here as a gameplay complaint rather than as a theorem.

**WHO TO FAVOUR IS A DESIGN DECISION**

**Favour the shooter** — full lag compensation. Shots land where the player aimed, and victims are occasionally hit after reaching cover. **Most competitive shooters choose this**, because aiming that does not work is intolerable to players in a way that being hit unfairly is not.

**Favour the target** — no compensation. Taking cover always works, and **shots feel unreliable**, which players attribute to the game being broken rather than to their own latency.

**Bound the rewind** — compensate up to perhaps 200 ms and no further. **This is the usual compromise**: it covers the great majority of honest connections while capping how much advantage a deliberately high-latency player can extract (§3's lag switch).

**And some games choose differently per mechanic** — favour the shooter for ranged weapons and the target for melee, because the perceptual expectations genuinely differ: a bullet that misses feels like a bug, while a sword swing that misses feels like a dodge. **There is no correct answer, only a stated one**, and the games that handle this best document their choice to players rather than leaving them to infer it from frustration.

### 3 Cheats, classified

| Cheat                 | What it exploits                                                                                               | Defence                                                                                                                                        |
|-----------------------|----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Speedhack             | <b>Client-controlled timing</b> — the client claims more time has passed than has.                             | <b>Server validates input rate against the wall clock. Solved completely.</b>                                                                  |
| Teleport, fly, noclip | <b>A server that accepts client-reported positions.</b>                                                        | <b>Server validates movement plausibility (§1). Solved completely.</b>                                                                         |
| Aimbot                | <b>Information the client legitimately has</b> — it must know where visible enemies are in order to draw them. | <b>Not solvable, only detectable.</b> The inputs are indistinguishable from a very good player's; only the statistical signature differs (§4). |
| Wallhack and ESP      | <b>State the server sent but the renderer chose not to draw.</b>                                               | <b>Do not send it</b> — see the callout. <b>Mostly solvable, and this is the highest-leverage defence available.</b>                           |
| Macros and scripting  | Input is input; a recoil-control macro produces legitimate inputs.                                             | <b>Undetectable in principle</b> at the protocol level. Policy and statistical detection only.                                                 |
| Lag switch            | <b>The disconnection ambiguity</b> (Module 07 §4) — deliberately inducing loss to exploit the grace period.    | Rate limits, bounded compensation (§2), and penalising the pattern rather than the individual event.                                           |

The split is sharp and is the most useful thing in this module: cheats that forge *outcomes* are completely solved by server validation, and cheats that exploit *information* or *reflexes* are not solvable at all. Knowing which category a reported cheat falls into tells you immediately whether you have a bug to fix or a trade-off to manage.

#### INFORMATION HIDING BEATS DETECTION

A wallhack works because the client was sent the position of a player it cannot see. The renderer declines to draw them; the memory contains them regardless, and reading memory on a machine you own is not difficult.

**So do not send it.** Server-side visibility determination — the relevance filtering of Module 09 §4, applied as a security measure rather than a bandwidth one — means the data is simply not present on the client to be extracted.

**This converts an arms race into an architecture decision**, which is an enormously better position: an arms race must be continuously funded forever, while an architecture decision is made once.

**It costs server CPU** (visibility must now be computed per-client, authoritatively), **and it is imperfect** — a player about to round a corner must be sent slightly before they are visible, or they will pop in late, so a small amount of soon-to-be-visible information inevitably leaks. **And it remains the single most effective anti-cheat measure available**, substantially more so than anything in §4, which is why it is worth the CPU.

## 4 Anti-cheat, honestly assessed

| Approach                          | What it catches                                                                                                   | What it costs                                                                                                                                                                                                                                         |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Server-side validation            | All outcome forgery (§1).                                                                                         | Server CPU. Non-negotiable — do this before anything else.                                                                                                                                                                                            |
| Information hiding                | Wallhacks and ESP (§3).                                                                                           | More server CPU. Do this too, and it pays for itself in bandwidth.                                                                                                                                                                                    |
| Statistical behavioural detection | Aimbots and macros, by their signature — reaction times, snap angles, and consistency that humans do not produce. | False positives ban real players, who are extremely difficult to re-acquire. Requires a human appeal path, which is an ongoing operational cost.                                                                                                      |
| Kernel-level anti-cheat           | Known client tampering, injected code, and debuggers.                                                             | Ring 0 code on every honest player's machine — a genuine security and privacy cost, a real crash and incompatibility risk, and a meaningful attack surface. It should be argued for explicitly, not assumed, and the objections to it are reasonable. |
| Player reporting                  | What humans notice, which is a surprisingly good signal.                                                          | Brigading and abuse; needs weighting by reporter reliability.                                                                                                                                                                                         |
| Hardware bans                     | Repeat offenders, for a while.                                                                                    | Spoofable, and causes collateral damage on shared or resold machines.                                                                                                                                                                                 |

### WHAT IS ACTUALLY ACHIEVABLE

**You can make outcome forgery impossible.** That is a solved problem, it is entirely within your control, and it requires no cooperation from the client. **Every game that is cheated by teleporting or by claimed damage has simply not done it.**

**You can make information-based cheats dramatically harder** by not sending the information (§3). Not perfectly, but enough to move the cheat from trivial to genuinely difficult.

**You cannot stop a sufficiently motivated attacker who controls the machine.** An external aimbot that reads the screen with a capture card on a second computer and moves the mouse with a microcontroller is **indistinguishable from a skilled player at every level you have access to** — the inputs are real mouse movements from real hardware. This is not a gap to be closed; it is a consequence of the attacker owning the endpoint.

**So the realistic goal is to raise the cost above what most people will pay, and to make what remains detectable by behaviour over time. 'Unhackable' is not on offer,** and claiming it is how a studio loses its players' trust — which is worth more than the marketing. **This is the same honesty principle as every other 'claiming honestly' section in this program:** state what you established and what you did not.

## Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

| Resource                                                                  | What it covers for this module                                                                                                                                                  |
|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Valve — Latency Compensating Methods (free)                               | <b>The §2 rewind technique</b> , documented by the people who popularised it, including the shot-around-a-corner consequence.                                                   |
| Glenn Fiedler — cheat prevention in networked games (free)                | The §1 principle, stated as a design rule.                                                                                                                                      |
| Riot Games — engineering blog on Valorant's netcode and anti-cheat (free) | <b>Server-side visibility (§3) as a deliberate anti-cheat architecture</b> , with the CPU cost discussed openly. The best available account of the information-hiding argument. |
| Hoglund & McGraw — Exploiting Online Games                                | The attacker's perspective on §3. Dated in specifics and accurate in structure.                                                                                                 |

## Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Audit your Module 09 project for every place a client reports an outcome** rather than an intent. Fix them.
- 2 Implement all five server-side validation checks.
- 3 **Write a speedhack** against your own unvalidated server, then defeat it with rate validation.
- 4 **Write a teleport hack**, then defeat it with movement validation.
- 5 Implement lag compensation with a one-second position history.
- 6 **Demonstrate the shot-around-a-corner case** from both players' points of view, and record both.
- 7 Bound the rewind to 200 ms and show what a 400 ms player now experiences.
- 8 **Implement server-side visibility determination** and verify that a hidden player's position is genuinely absent from the client's received packets.
- 9 Measure the server CPU cost of that visibility determination against player count.
- 10 **Write one page assessing your own game's cheat resistance honestly** — what is solved, what is mitigated, and what is not addressed.

## Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the intent-versus-outcome rule and give two examples of each.
- 2 Name five server-side validation checks.
- 3 Explain lag compensation and the problem it solves.
- 4 Why do both players experience the game correctly and still disagree?
- 5 Give three ways to resolve who is favoured.
- 6 Classify six cheats by what they exploit, and say which are solvable.
- 7 Why does information hiding beat detection?
- 8 Give six anti-cheat approaches and the cost of each.
- 9 What is actually achievable, and what is not?